New Relic APM — Day 5 Lab Guide

Hands-on labs for *Advanced NRQL, integrations and capstone* · easyTravel, Spring PetClinic and a kind cluster on Linux · NRQL, NerdGraph, workflows and service levels

How to use this guide

Each lab matches a module of the New Relic APM — Day 5 Study Guide: read the module, then do its lab. The labs use the data and components from Days 1–4.

Lab	Title	Module	Time
5.1	Answer production questions with advanced NRQL	Module 13	60 min
5.2	Integrations: Slack and incident tools, CI/CD deployment tracking, Kubernetes, correlation	Module 14	60 min
5.3	Capstone: end-to-end setup, service levels, error budget, reporting and recommendations	Module 15	75 min + presentations

Before you start

- Days 1–4 are complete: easyTravel, PetClinic, PostgreSQL, the infrastructure agent, the headless browsers (`pc-browser`) and the synthetics job manager are running; `pcload.sh` is running.
- `~/nr-lab/.env` holds `NEW_RELIC_LICENSE_KEY`, the user key `NEW_RELIC_API_KEY` (Day 2) and `NEW_RELIC_ACCOUNT_ID`. Add the account ID if it is missing: `echo "NEW_RELIC_ACCOUNT_ID=1234567" >> ~/nr-lab/.env` with your own number.
- For Lab 5.2 you need a Slack workspace where you may install apps — a free workspace created for the course is fine. PagerDuty and ServiceNow steps are optional.
- The VM needs about **2 GB more memory and 4 GB more disk** for the kind cluster.

```
cd ~/nr-lab
docker ps --format "table {{.Names}}\t{{.Status}}" | head -20
grep -c "NEW_RELIC_API_KEY\|NEW_RELIC_ACCOUNT_ID" ~/nr-lab/.env
free -h | head -2; df -h ~ | tail -1
```

Conventions

- `$` lines run on the lab VM as `trainee`, from `~/nr-lab` unless a step says otherwise.
- **Expected output** pictures show terminal output; **Expected view** pictures are redrawn screens — your numbers, names and layout will differ.
- Run every NRQL query in **Query your data** for the last 3 hours unless the step says otherwise, and save the useful ones with *Add to dashboard* — Lab 5.3 reuses them.

Lab 5.1 — Answer production questions with advanced NRQL

Goal: answer seven questions an incident commander or product owner would ask about the easyTravel and PetClinic services, each with one query — using `filter()` ratios, `rate()`, nested aggregation, time comparisons, a funnel, percentiles and histograms, `CASES`, `buckets()`, `capture()` and a subquery.

Time: 60 minutes. **Module:** 13.

Step 1 — Make the browser users behave like people

The Day 4 browsers complete every journey, so a funnel would show no drop-offs. Replace their script with one where some visitors leave early or find nothing:

```
cat > ~/nr-lab/rum/pcbrowse.mjs <<'EOF'
import { chromium, devices } from 'playwright';

const BASE = process.env.BASE || 'http://localhost:8090';
const profiles = ['Desktop Chrome', 'Desktop Edge', 'Pixel 7', 'iPhone 13', 'Galaxy S9+'];
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const leaves = (p) => Math.random() < p;          // this visitor gives up here
const browser = await chromium.launch();

while (true) {
  const name = profiles[Math.floor(Math.random() * profiles.length)];
  const context = await browser.newContext({ ...devices[name] });
  const page = await context.newPage();
  let reached = 'home';
  try {
    await page.goto(BASE + '/');
    if (!leaves(0.25)) {
      await page.goto(BASE + '/owners/find'); reached = 'find';
      if (!leaves(0.15)) {
        const typo = leaves(0.2);                  // one in five types a name that does not
exist
        await page.fill('#lastName', typo ? 'Lsat' + Math.floor(Math.random() * 999) : 'Last'
+ Math.floor(Math.random() * 50000));
        await page.click('#search-owner-form button[type="submit"]');
        await page.waitForLoadState('load'); reached = typo ? 'no results' : 'results';
        const first = page.locator('#owners a').first();
        if (!typo && !leaves(0.3) && await first.count()) {
          await first.click(); await page.waitForLoadState('load'); reached = 'details';
        }
      }
    }
    await sleep(3000);
    console.log(new Date().toISOString(), name, 'reached:', reached);
  } catch (e) {
    console.log(new Date().toISOString(), name, 'failed:', e.message);
  }
  await context.close();
  await sleep(3000);
}
EOF
docker restart pc-browser && sleep 120 && docker logs --tail 6 pc-browser
```

Lab 5.1 · Step 1 · Browser visitors with drop-offs — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ docker restart pc-browser && sleep 120 && docker logs --tail 6
                         pc-browser
pc-browser
2026-10-29T13:12:04.551Z Pixel 7 reached: details
2026-10-29T13:12:12.093Z Desktop Chrome reached: home
2026-10-29T13:12:21.870Z iPhone 13 reached: no results
2026-10-29T13:12:31.402Z Desktop Edge reached: details
2026-10-29T13:12:38.914Z Galaxy S9+ reached: find
2026-10-29T13:12:48.226Z Desktop Chrome reached: results
wait about 20 minutes before running the funnel in Question 4
```

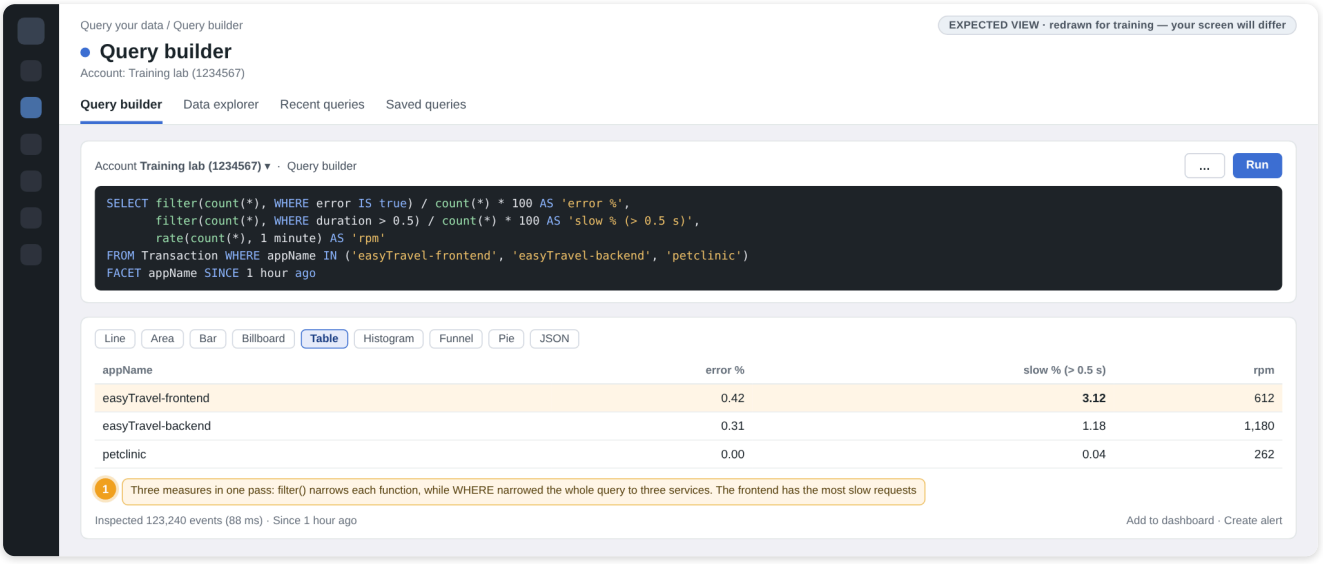
Expected output — the browsers restarted with the new script; some visitors stop at home, find or no results

Let them run for 20 minutes before Question 4. Start with the questions that use server data.

Question 1 — Which service is in the worst shape right now?

One query, three measures side by side, thanks to `filter()` and `rate()`:

```
SELECT filter(count(*), WHERE error IS true) / count(*) * 100 AS 'error %',
       filter(count(*), WHERE duration > 0.5) / count(*) * 100 AS 'slow % (> 0.5 s)',
       rate(count(*), 1 minute) AS 'rpm'
FROM Transaction WHERE appName IN ('easyTravel-frontend', 'easyTravel-backend', 'petclinic')
FACET appName SINCE 1 hour ago
```

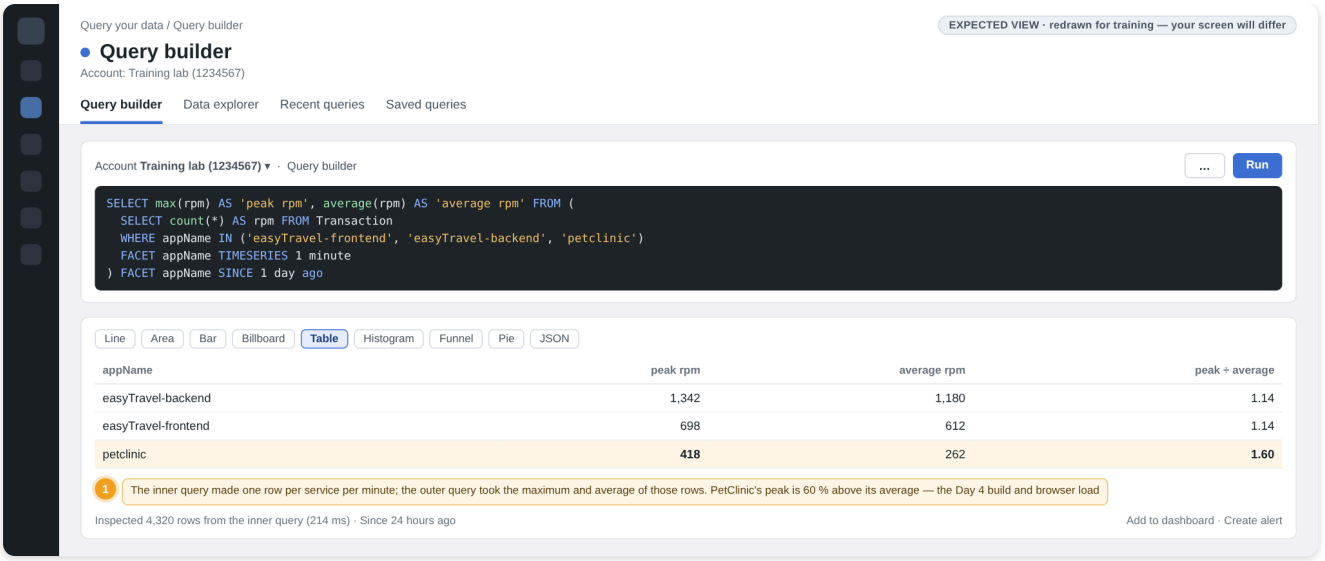


Expected view — error rate, slow rate and throughput per service in one table, from filter() and rate()

Try the same with `WHERE error IS true` instead of `filter()`: the percentage becomes 100 % — `WHERE` removed everything else before counting.

Question 2 — What was each service's busiest minute today?

```
SELECT max(rpm) AS 'peak rpm', average(rpm) AS 'average rpm' FROM (
  SELECT count(*) AS rpm FROM Transaction
  WHERE appName IN ('easyTravel-frontend', 'easyTravel-backend', 'petclinic')
  FACET appName TIMESERIES 1 minute
) FACET appName SINCE 1 day ago
```

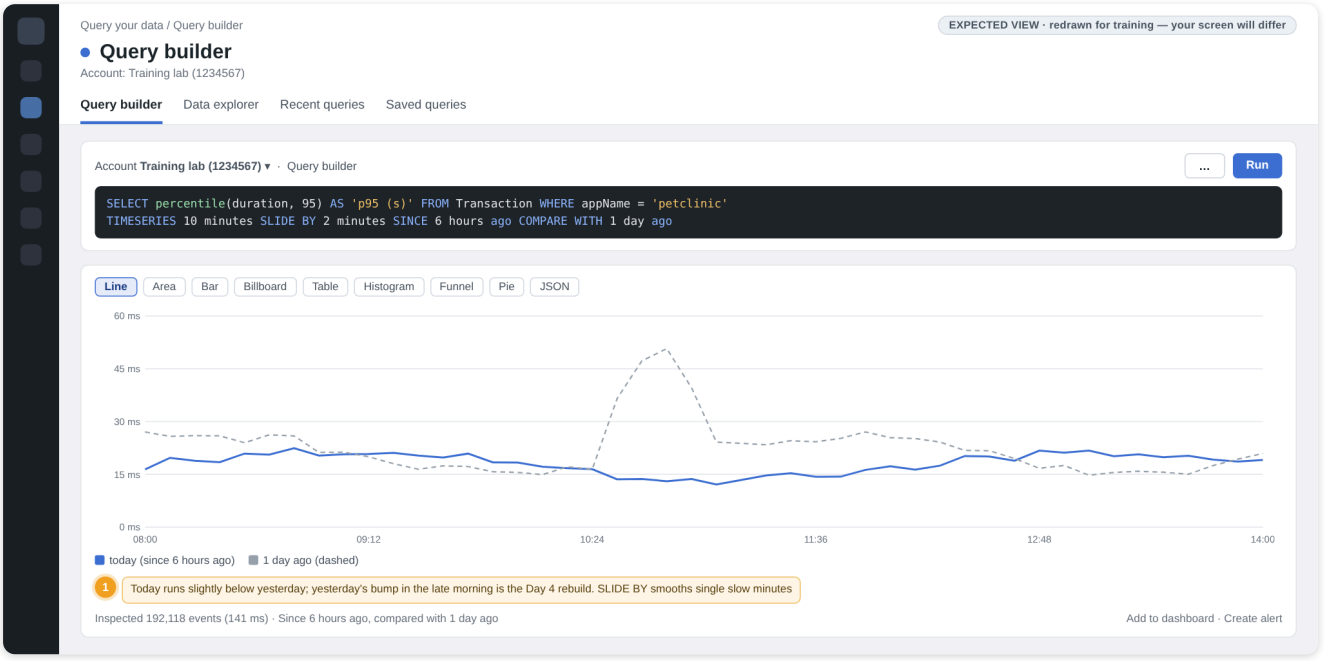


Expected view — nested aggregation: peak and average requests per minute per service, from per-minute rows

Capacity is planned for the peak, not the average: the ratio between the two columns is the headroom a service needs.

Question 3 — Is today worse than yesterday?

```
SELECT percentile(duration, 95) AS 'p95 (s)' FROM Transaction WHERE appName = 'petclinic'
TIMESERIES 10 minutes SLIDE BY 2 minutes SINCE 6 hours ago COMPARE WITH 1 day ago
```



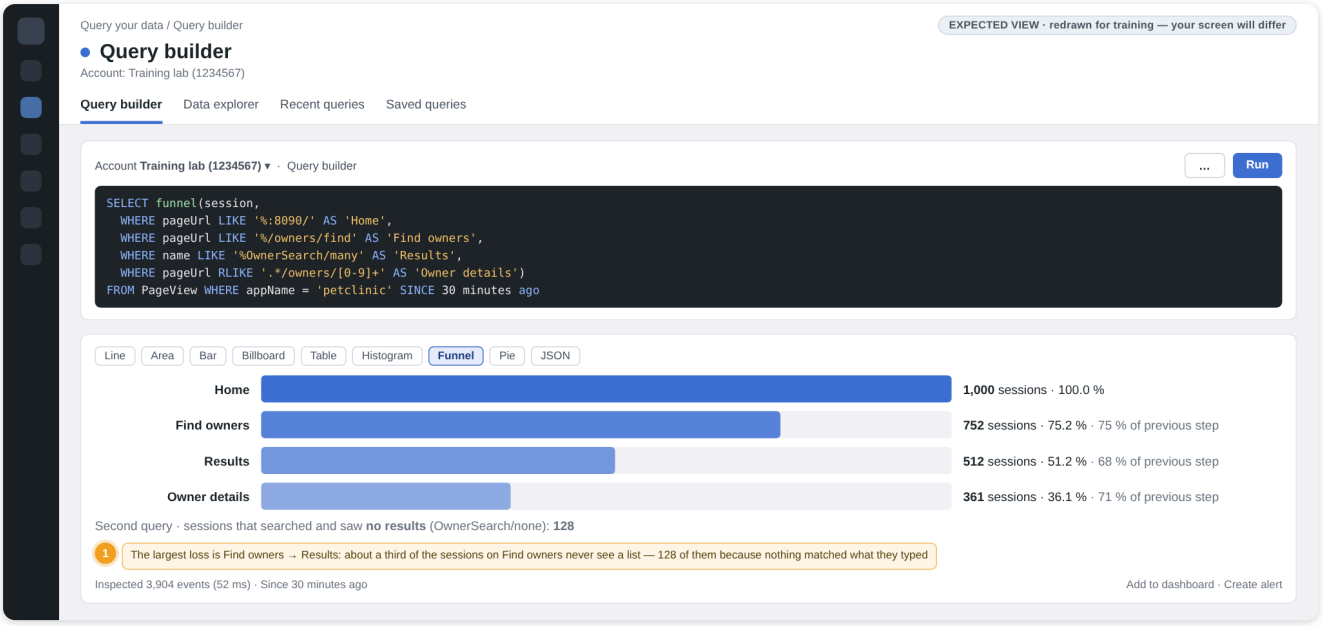
Expected view — p95 for the last six hours as a smoothed line, with yesterday's same hours dashed

`SLIDE BY` overlaps the windows, so single slow minutes do not dominate. In production compare with `1 week ago` — the same weekday — once you have the data.

Question 4 — Where do users drop out of the find-owner journey?

After 20 minutes of the new browser script:

```
SELECT funnel(session,
  WHERE pageUrl LIKE '%:8090/' AS 'Home',
  WHERE pageUrl LIKE '%/owners/find' AS 'Find owners',
  WHERE name LIKE '%OwnerSearch/many' AS 'Results',
  WHERE pageUrl RLIKE '.*/owners/[0-9]+' AS 'Owner details')
FROM PageView WHERE appName = 'petclinic' SINCE 30 minutes ago
```



Expected view — the funnel: sessions at each step of the journey and the drop between steps

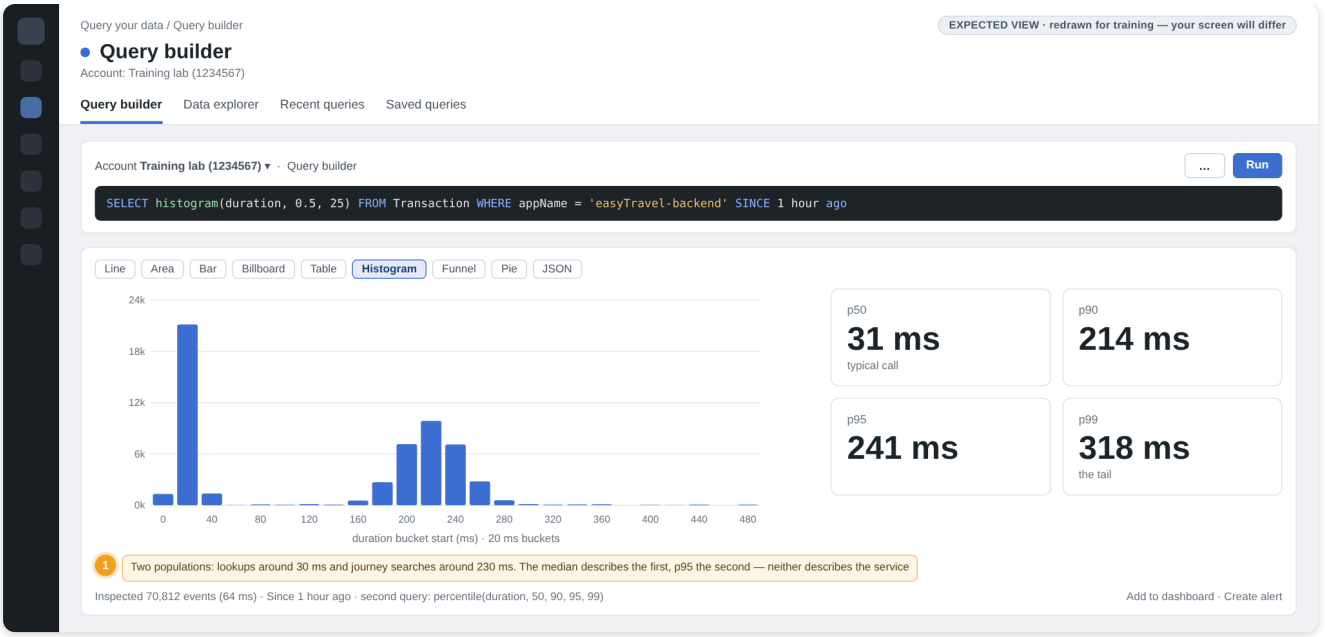
```
-- Where did the sessions that searched but saw no results end up?
SELECT uniqueCount(session) FROM PageView WHERE appName = 'petclinic' AND name LIKE
'%OwnerSearch/none' SINCE 30 minutes ago
```

The script's probabilities are known, so you can check the funnel: about 75 % of sessions reach *Find owners*; of those, about 68 % see results; of those, about 70 % open an owner. The biggest loss in this journey is visitors who search but find nothing — an argument for "did you mean…?" suggestions.

Question 5 — How slow is slow on the easyTravel backend?

```
SELECT percentile(duration, 50, 90, 95, 99) FROM Transaction WHERE appName = 'easyTravel-
backend' SINCE 1 hour ago

SELECT histogram(duration, 0.5, 25) FROM Transaction WHERE appName = 'easyTravel-backend'
SINCE 1 hour ago
```



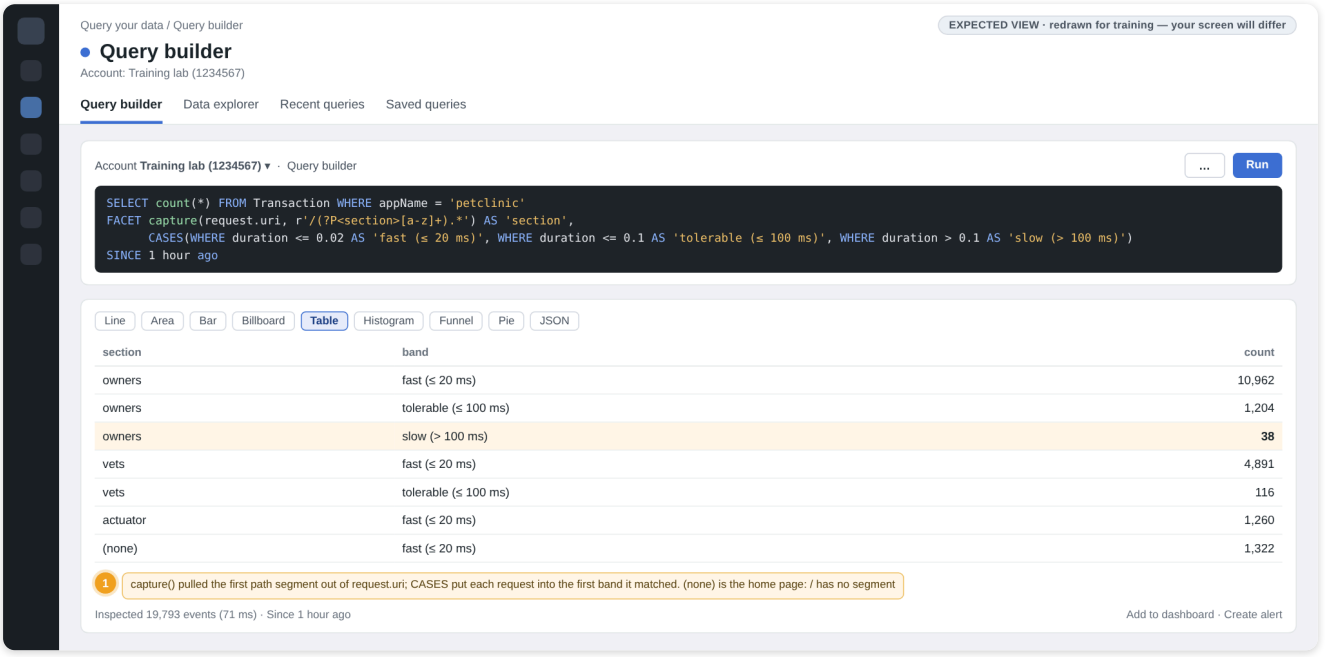
Expected view — the backend's latency histogram with two peaks, and its percentiles

The histogram usually shows two groups: fast calls (location and authentication lookups) and slower journey searches. One average or one percentile describes neither. Find out which transactions form each peak:

```
SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-backend'
FACET name, buckets(duration, 0.5, 5) SINCE 1 hour ago LIMIT 20
```

Question 6 — How many requests are fast, tolerable or slow, per section of the site?

```
SELECT count(*) FROM Transaction WHERE appName = 'petclinic'
FACET capture(request.uri, r'/(?P<section>[a-z]+).*') AS 'section',
      CASES(WHERE duration <= 0.02 AS 'fast (≤ 20 ms)', WHERE duration <= 0.1 AS 'tolerable
(≤ 100 ms)', WHERE duration > 0.1 AS 'slow (> 100 ms)')
SINCE 1 hour ago
```



Expected view — PetClinic requests per URL section and experience band, from capture() and FACET CASES

Reorder the `CASES` so that `duration > 0.1` comes first and `duration <= 0.1` second: nothing changes. Now put `duration <= 0.1` before `duration <= 0.02`: the *fast* band empties, because every fast request matched the wider case first.

Question 7 — Which traces contain slow owner queries?

```
SELECT traceId, name, duration * 1000 AS 'ms' FROM Transaction
WHERE appName = 'petclinic' AND traceId IN (
  FROM Span SELECT uniques(trace.id)
  WHERE appName = 'petclinic' AND name LIKE 'Datastore/statement/Postgres/owners%' AND
duration.ms > 20 LIMIT MAX)
SINCE 3 hours ago LIMIT 20
```

The inner query finds the traces from `Span` events (`trace.id`); the outer query shows the transactions they belong to (`traceId` on `Transaction`). Open one trace ID in **Distributed tracing**. (With the Day 3 index in place there may be none — which is itself an answer.)

Check your work

- ☐ You answered the seven questions and saved at least Questions 1, 2, 4 and 5 to a dashboard page named `Analysis`.
- ☐ You can explain why `WHERE` and `filter()` give different results, and why the order of `CASES` matters.

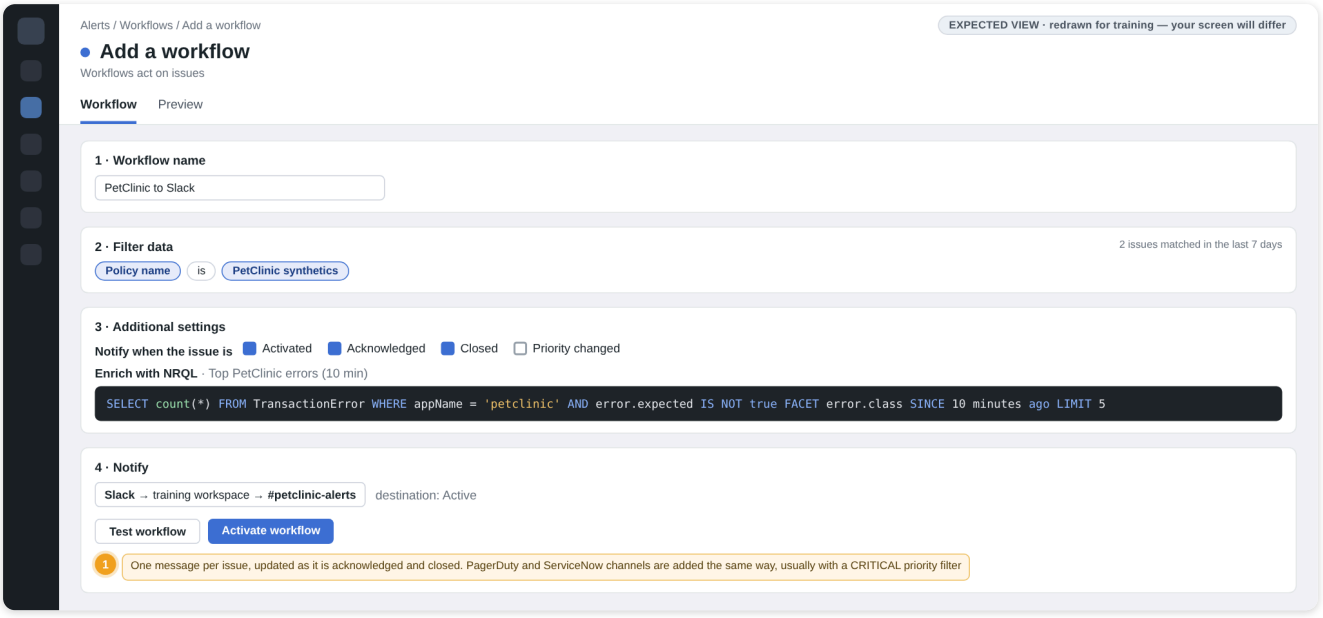
Lab 5.2 — Integrations: Slack and incident tools, CI/CD deployment tracking, Kubernetes, correlation

Goal: send PetClinic issues to a Slack channel (and optionally PagerDuty and ServiceNow), record releases from a deployment script, add the Kubernetes integration on a local cluster, and see how one combined problem appears as one correlated issue with its deployment and one notification.

Time: 60 minutes. **Module:** 14.

Step 1 — Slack destination and workflow

1. In Slack, create the channel `#petclinic-alerts`.
2. In New Relic: **Alerts** → **Destinations** → **Slack**, connect your workspace and allow the New Relic app. The destination appears as *Active*. Invite the app to the channel if Slack asks.
3. **Alerts** → **Workflows** → **Add a workflow**, name `PetClinic to Slack`:
   - **Filter:** policy name is `PetClinic synthetics` (Day 4) — this policy will also hold the APM condition from Step 4.
   - **Notify when:** activated, acknowledged, closed.
   - **Enrich:** `SELECT count(*) FROM TransactionError WHERE appName = 'petclinic' AND error.expected IS NOT true FACET error.class SINCE 10 minutes ago LIMIT 5`
   - **Channel:** Slack → your workspace → `#petclinic-alerts`.
4. **Test workflow**: a test message arrives in the channel. **Activate**.



Expected view — the PetClinic to Slack workflow: policy filter, triggers, NRQL enrichment and the Slack channel

Optional — PagerDuty. In a PagerDuty account (a free developer or trial account works), create a service `petclinic` with an **Events API v2** integration and copy its integration key. In New Relic: **Destinations** → **PagerDuty** → **service integration**, paste the key. Add a second workflow `PetClinic page` with the filter *policy is* `PetClinic synthetics` *AND priority equals* `CRITICAL` and the PagerDuty channel.

Optional — ServiceNow. With a ServiceNow instance (a free personal developer instance works), create an integration user with the incident roles. In New Relic: **Destinations** → **ServiceNow**, enter the instance URL

and the user. In a workflow channel choose *create incident* and map at least *short description*, *urgency*, *assignment group* and *description* (with the issue link).

Step 2 — A deployment script that records the release

The script builds PetClinic, sets the version the agent reports, deploys, runs a smoke test and only then records the deployment with NerdGraph:

```
mkdir -p ~/nr-lab/ci
cat > ~/nr-lab/ci/deploy.sh <<'EOF'
#!/usr/bin/env bash
# Minimal CI/CD pipeline for PetClinic: build -> deploy -> smoke test -> record deployment
set -euo pipefail
VERSION="$1"; CHANGELOG="${2:-no changelog}"
set -a; source ~/nr-lab/.env; set +a
DC="docker compose -f $HOME/nr-lab/petclinic-compose.yml --env-file $HOME/nr-lab/.env"
NR_VERSION=$(python3 -c "import zipfile,os; m=zipfile.ZipFile(os.path.expanduser('~/nr-
lab/agent/petclinic/newrelic.jar')).read('META-INF/MANIFEST.MF').decode(); print([l.split(':
')[1].strip() for l in m.splitlines() if l.startswith('Implementation-Version')][0])")

echo "== build $VERSION"
docker run --rm -v "$HOME/nr-lab/petclinic-src":/src -v "$HOME/.m2":/root/.m2 -w /src
maven:3.9-eclipse-temurin-17 \
  mvn -q -DskipTests -Dspring-javaformat.skip=true -Dcheckstyle.skip -
Dnewrelic.version="$NR_VERSION" package
cp ~/nr-lab/petclinic-src/target/spring-petclinic-*.jar ~/nr-lab/petclinic.jar

echo "== deploy $VERSION"
sed -i "s/NEW_RELIC_METADATA_SERVICE_VERSION: .*/NEW_RELIC_METADATA_SERVICE_VERSION:
\"$VERSION\"/" ~/nr-lab/petclinic-compose.yml
$DC up -d --force-recreate petclinic > /dev/null
for i in $(seq 1 30); do curl -sf -o /dev/null http://localhost:8090/actuator/health &&
break; sleep 5; done

echo "== smoke test"
curl -sf -o /dev/null "http://localhost:8090/owners?lastName=Davis" || { echo "smoke test
FAILED - no marker recorded"; exit 1; }
echo "owner search OK"

echo "== record deployment"
API=https://api.newrelic.com/graphql
Q="{ actor { entitySearch(query: \"name = 'petclinic' AND domain = 'APM' AND type =
'APPLICATION'\") { results { entities { guid } } } } }"
GUID=$(jq -n --arg q "$Q" '{query: $q}' | curl -s "$API" -H 'Content-Type: application/json'
-H "API-Key: $NEW_RELIC_API_KEY" -d @- \
  | jq -r '.data.actor.entitySearch.results.entities[0].guid')
M="mutation { changeTrackingCreateDeployment(deployment: { entityGuid: \"$GUID\", version:
\"$VERSION\", changelog: \"$CHANGELOG\", user: \"$USER\", deploymentType: BASIC, description:
\"deploy.sh\" }) { deploymentId timestamp } }"
jq -n --arg q "$M" '{query: $q}' | curl -s "$API" -H 'Content-Type: application/json' -H
"API-Key: $NEW_RELIC_API_KEY" -d @- | jq -c .
EOF
chmod +x ~/nr-lab/ci/deploy.sh
~/nr-lab/ci/deploy.sh 4.1.0 "Owner search telemetry and browser monitoring (Day 4)"
```

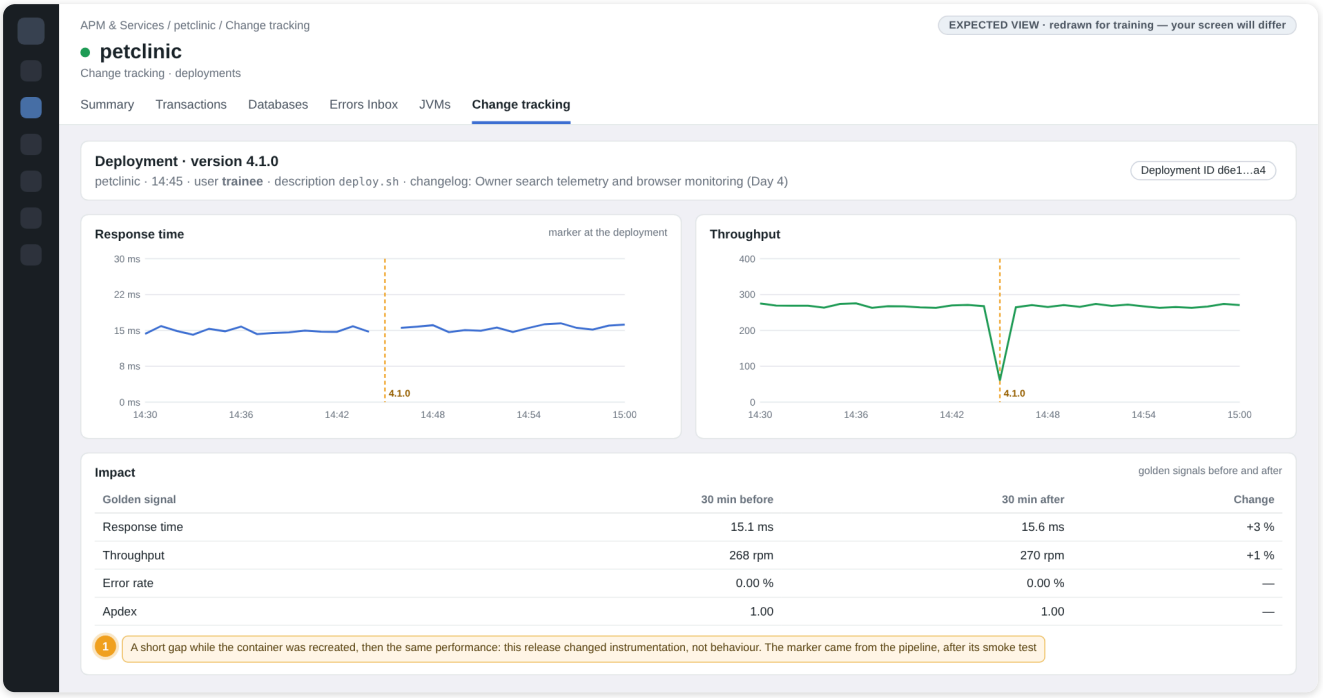
Lab 5.2 · Step 2 · deploy.sh — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ ~/nr-lab/ci/deploy.sh 4.1.0 "Owner search telemetry and browser
                         monitoring (Day 4)"
== build 4.1.0
== deploy 4.1.0
== smoke test
owner search OK
== record deployment
{"data":{"changeTrackingCreateDeployment":{"deploymentId":"d6e1c0f2-6b1e-4a7e-9c55-2f0a8b1e44a4"
    ,"timestamp":1793272814000}}}
trainee@nr-lab:~/nr-lab$ docker exec petclinic env | grep SERVICE_VERSION
NEW_RELIC_METADATA_SERVICE_VERSION=4.1.0
the marker is recorded only after the smoke test passed
```

Expected output — the pipeline builds, deploys version 4.1.0, passes the smoke test and records the deployment

The marker is created only after the smoke test passes, and with the same version the agent now reports in `tags.service.version`. EU accounts use `api.eu.newrelic.com`.

Open **petclinic** → **Summary** (deployment marker on the charts) and **Change tracking** (or *Deployments*): the release with its version, user and changelog, and before/after response time, throughput and errors.



Expected view — change tracking for petclinic: the 4.1.0 deployment with its changelog and before/after impact

```
SELECT * FROM Deployment SINCE 1 day ago
```

In a real pipeline the same `curl` runs as the last job of the deploy stage, with the user key as a CI secret; GitHub Actions can use `newrelic/deployment-marker-action` instead (study guide, 14.2).

Step 3 — Kubernetes: a kind cluster with the New Relic bundle

```
# Tools (skip any you already have)
curl -sLo kind https://kind.sigs.k8s.io/dl/v0.24.0/kind-linux-amd64 && chmod +x kind && sudo
mv kind /usr/local/bin/
curl -sLO https://dl.k8s.io/release/v1.31.0/bin/linux/amd64/kubectl && chmod +x kubectl &&
```

```
sudo mv kubectl /usr/local/bin/
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Cluster and the New Relic Kubernetes integration
kind create cluster --name nr-lab
set -a; source ~/nr-lab/.env; set +a
helm repo add newrelic https://helm-charts.newrelic.com && helm repo update
helm upgrade --install newrelic-bundle newrelic/nri-bundle \
  --namespace newrelic --create-namespace \
  --set global.licenseKey="$NEW_RELIC_LICENSE_KEY" \
  --set global.cluster=nr-lab-kind \
  --set newrelic-infrastructure.privileged=true \
  --set kube-state-metrics.enabled=true \
  --set kubeEvents.enabled=true \
  --set global.lowDataMode=true

# Two workloads: a healthy one and one that keeps crashing
kubectl create deployment web --image=nginx:1.27 --replicas=2
kubectl create deployment broken --image=busybox:1.36 -- sh -c "echo starting; sleep 5; exit
1"
sleep 120; kubectl get pods -A
```

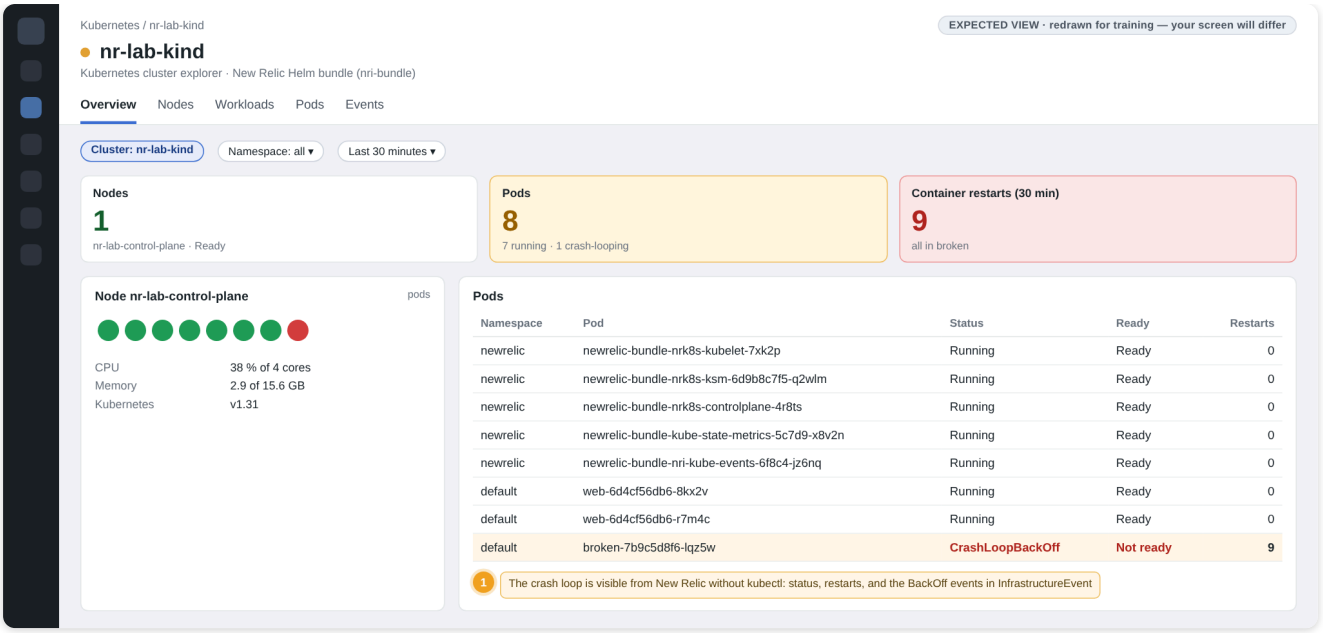
Lab 5.2 · Step 3 · kind and the New Relic bundle — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ kind create cluster --name nr-lab
Creating cluster "nr-lab" ...
 ✓ Ensuring node image (kindest/node:v1.31.0)
 ✓ Starting control-plane
Set kubectl context to "kind-nr-lab"
trainee@nr-lab:~/nr-lab$ helm upgrade --install newrelic-bundle newrelic/nri-bundle --namespace
                         newrelic --create-namespace --set
                         global.licenseKey="$NEW_RELIC_LICENSE_KEY" --set
                         global.cluster=nr-lab-kind …
Release "newrelic-bundle" does not exist. Installing it now.
STATUS: deployed
trainee@nr-lab:~/nr-lab$ kubectl get pods -A | grep -v kube-system
NAMESPACE   NAME                                              READY   STATUS             RESTARTS
default     broken-7b9c5d8f6-lqz5w                            0/1     CrashLoopBackOff   4
default     web-6d4cf56db6-8kx2v                              1/1     Running            0
default     web-6d4cf56db6-r7m4c                              1/1     Running            0
newrelic    newrelic-bundle-kube-state-metrics-5c7d9-x8v2n    1/1     Running            0
newrelic    newrelic-bundle-nri-kube-events-6f8c4-jz6nq       2/2     Running            0
newrelic    newrelic-bundle-nrk8s-controlplane-4r8ts          2/2     Running            0
newrelic    newrelic-bundle-nrk8s-ksm-6d9b8c7f5-q2wlm         2/2     Running            0
newrelic    newrelic-bundle-nrk8s-kubelet-7xk2p               2/2     Running            0
```

Expected output — the kind cluster with the New Relic bundle pods running, two web pods, and a crash-looping broken pod

The license key reaches the cluster as a Helm value, which Helm stores in a Kubernetes secret. In production, create the secret yourself and point the chart at it (`global.customSecretName`) so the key never appears in shell history or CI logs.

Open **Kubernetes** → **nr-lab-kind** (the cluster explorer): nodes, namespaces, deployments and pods, with `broken` in a restart loop.



Expected view — the cluster explorer: one node, pods by namespace, and the broken deployment restarting

```
SELECT latest(status), latest(isReady) FROM K8sPodSample WHERE clusterName = 'nr-lab-kind'
FACET namespaceName, podName SINCE 10 minutes ago

SELECT max(restartCount) FROM K8sContainerSample WHERE clusterName = 'nr-lab-kind' FACET
deploymentName, containerName SINCE 30 minutes ago

SELECT count(*) FROM InfrastructureEvent WHERE clusterName = 'nr-lab-kind' FACET
event.reason, event.involvedObject.name SINCE 30 minutes ago
```

AWS and Azure work the same way at the account level: an IAM role or a service principal with read access, and the data arrives as entities related to what runs on them. The lab has no cloud account, so they are covered in the study guide only.

Step 4 — One problem, one issue: correlation

Add an APM condition to the PetClinic policy so that one problem trips conditions from two data sources:

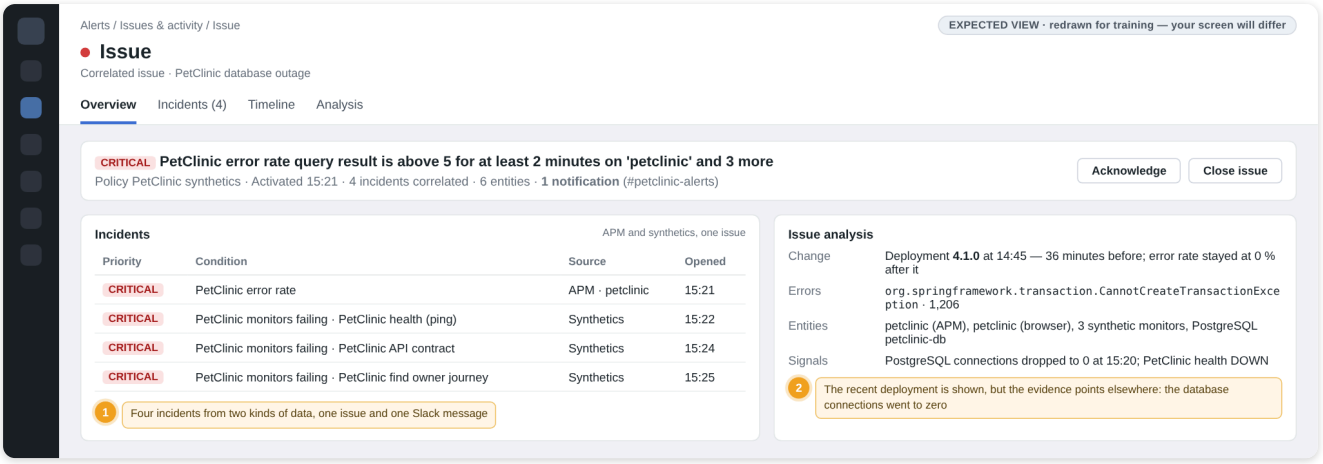
1. In policy `PetClinic synthetics`, add the NRQL condition `PetClinic error rate`: `SELECT percentage(count(*), WHERE error IS true) FROM Transaction WHERE appName = 'petclinic'`, critical above 5 for at least 2 minutes.
2. In the policy settings, check that **Correlate and suppress noise** is on.
3. Break the database for five minutes:

```
date +%H:%M; docker stop petclinic-db
sleep 300
docker start petclinic-db && sleep 60; date +%H:%M
curl -s -o /dev/null -w "petclinic health: HTTP %{http_code}\n"
http://localhost:8090/actuator/health
```

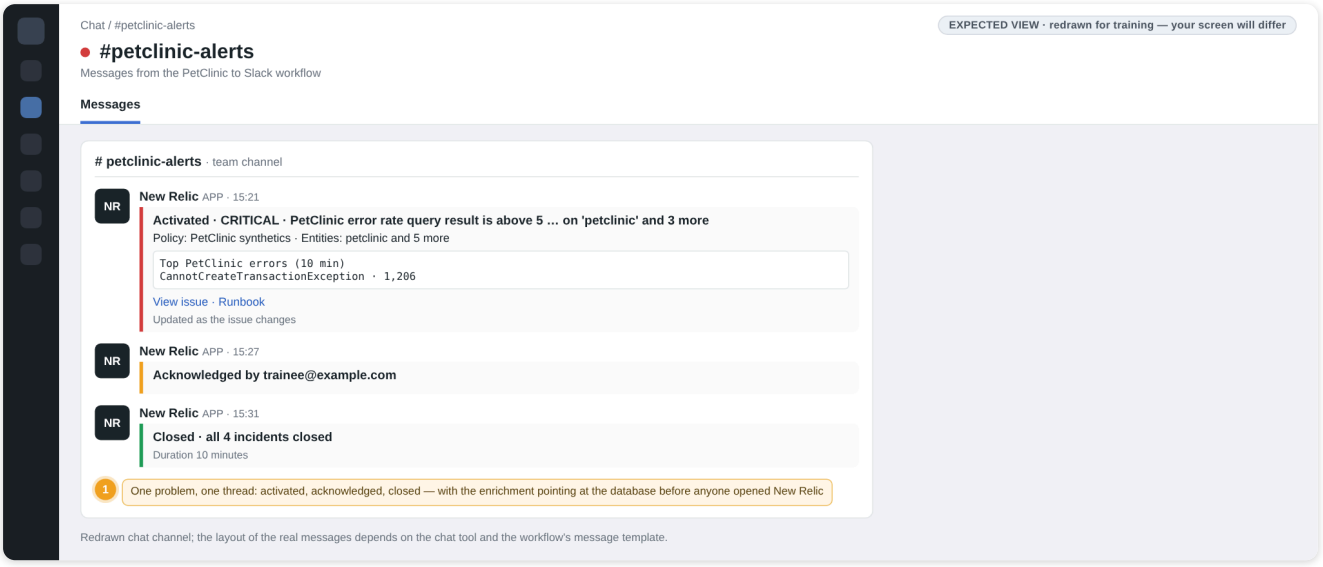
While the database is down, PetClinic returns errors, its health endpoint reports `DOWN`, and the synthetics ping, API and journey monitors fail. Open the issue in **Alerts → Issues & activity**:

- Several incidents — the error rate condition and the synthetic monitors — in **one issue**.

- The **4.1.0 deployment** from Step 2 listed as a recent change: was it the cause? (No — the timing and the errors say otherwise; the database stopped.)
- The issue's analysis: impacted entities, related signals and error groups pointing at the database connection.
- In `#petclinic-alerts`: **one** message for the issue, updated when it is acknowledged and closed — not one per incident.



Expected view — the correlated issue: incidents from APM and synthetics, the recent deployment, and the analysis pointing at the database



Expected view — the Slack channel: one message for the issue with its enrichment, then the closed update

```
SELECT count(*) FROM NrAiIncident WHERE policyName = 'PetClinic synthetics' AND event =
'open' SINCE 30 minutes ago
SELECT uniqueCount(issueId) FROM NrAiIssue SINCE 30 minutes ago
```

Several incidents, one issue, one notification: that ratio is the number to watch when you tune correlation.

Check your work

- ☐ The Slack workflow is active and the channel received the test message and the issue.
- ☐ `deploy.sh` recorded version 4.1.0, and the marker shows on the petclinic charts.
- ☐ The kind cluster reports to New Relic; you can find the crash-looping pod in NRQL and in the cluster explorer.

- ☐ The database outage produced one issue with several incidents, and PetClinic is healthy again.

Lab 5.3 — Capstone: end-to-end setup, service levels, error budget, reporting and recommendations

Goal: treat the lab system as a customer's microservice application. Audit the setup, define service levels for the critical journeys with an error budget and burn-rate alerts, add service-level, business and executive pages to the dashboard, write a one-page performance report, and present three improvement recommendations with evidence.

Time: 75 minutes, then 5 minutes of presentation per person or team. **Module:** 15.

Step 1 — Audit the end-to-end setup

Check each layer from the study guide (15.1) and record *done / gap* in your notes:

```
-- 1. Services, versions and distributed tracing
SELECT uniques(tags.service.version), percentage(count(*), WHERE traceId IS NOT NULL) AS
'traced %'
FROM Transaction FACET appName SINCE 1 hour ago

-- 2. Logs in context
SELECT count(*) FROM Log WHERE entity.name IS NOT NULL FACET entity.name SINCE 1 hour ago

-- 3. Browser and synthetics
SELECT count(*) FROM PageView FACET appName SINCE 1 hour ago
SELECT latest(result) FROM SyntheticCheck FACET monitorName SINCE 30 minutes ago

-- 4. Hosts, containers, databases, Kubernetes
SELECT uniqueCount(name) FROM ContainerSample SINCE 10 minutes ago
SELECT latest(db.connections) FROM PostgresqlDatabaseSample SINCE 10 minutes ago
SELECT uniqueCount(podName) FROM K8sPodSample FACET clusterName SINCE 10 minutes ago

-- 5. Business context
SELECT count(*) FROM OwnerSearch FACET outcome SINCE 1 hour ago
```

Then tag the services so that dashboards, workloads and alert routing can use the tags (repeat for each service GUID; find GUIDs with the entity search from Lab 5.2):

```
mutation { taggingAddTagsToEntity(guid: "ENTITY_GUID",
  tags: [{ key: "team", values: ["platform"] }, { key: "env", values: ["lab"] }, { key:
"tier", values: ["1"] }]) {
  errors { message } } }
```

Step 2 — Choose the critical journeys and their targets from data

Journey	Service	SLI type	Good event
Book a trip	easyTravel-frontend	Success	No unexpected error
Book a trip	easyTravel-frontend	Latency	Faster than a threshold you choose
Find an owner	petclinic	Latency	Owner search faster than a threshold you choose
Use PetClinic	petclinic	Success	No unexpected error

Look at what the services actually achieve before you set targets:

```
SELECT percentile(duration, 95) AS 'p95 (s)',
       percentage(count(*), WHERE error IS false) AS 'success %'
FROM Transaction WHERE appName IN ('easyTravel-frontend', 'petclinic') AND transactionType =
'Web'
FACET appName SINCE 7 days ago

SELECT percentage(count(*), WHERE duration < 1) AS 'frontend < 1 s %' FROM Transaction
WHERE appName = 'easyTravel-frontend' AND transactionType = 'Web' SINCE 7 days ago

SELECT percentage(count(*), WHERE duration < 0.1) AS 'owner search < 100 ms %' FROM
Transaction
WHERE appName = 'petclinic' AND name LIKE '%OwnerSearch%' SINCE 7 days ago
```

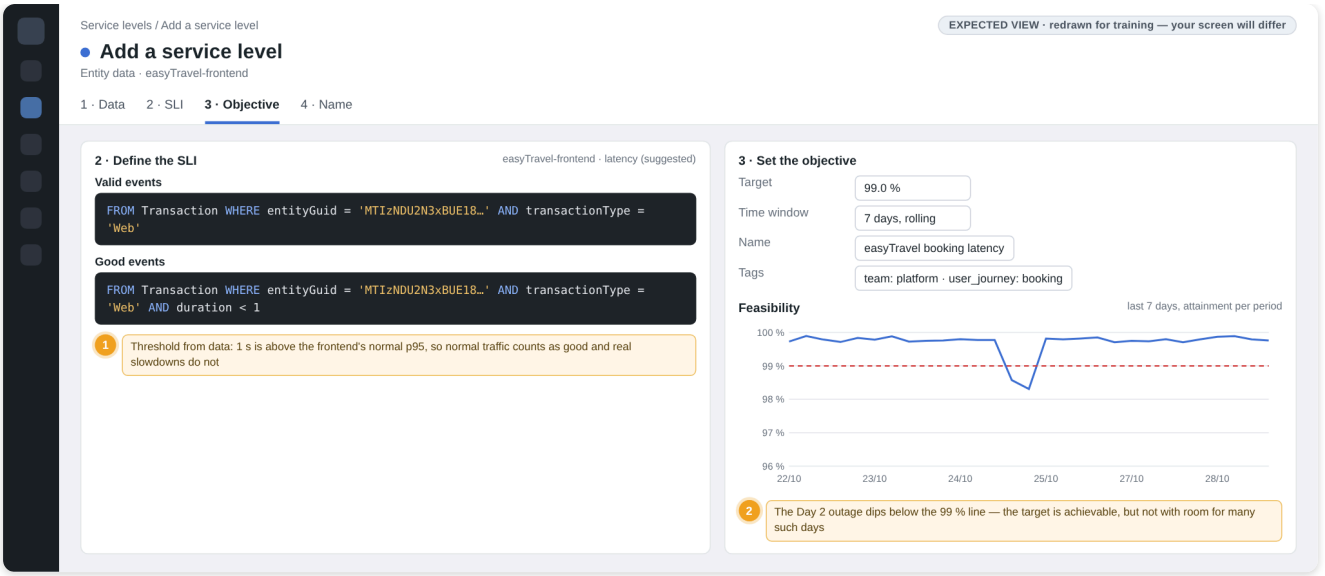
The Day 2 and Day 3 outages are in this data — which is right: targets must survive real weeks. Pick targets a little below what the services achieved (for example 99 % rather than 99.6 %) and a 7-day rolling window, because the lab has only a few days of history. In production use 28 days.

Step 3 — Create the service levels

1. **Service levels → + Add a service level**. Select the entity `easyTravel-frontend`; choose the suggested **success** SLI (valid: its transactions; bad: `TransactionError` where `error.expected != true`); target from Step 2, 7-day window; name `easyTravel booking success`; tags `team: platform`, `user_journey: booking`.
2. Repeat for the suggested **latency** SLI of `easyTravel-frontend`, with your threshold.
3. Create the owner search latency SLI with NerdGraph (it needs a custom `WHERE`):

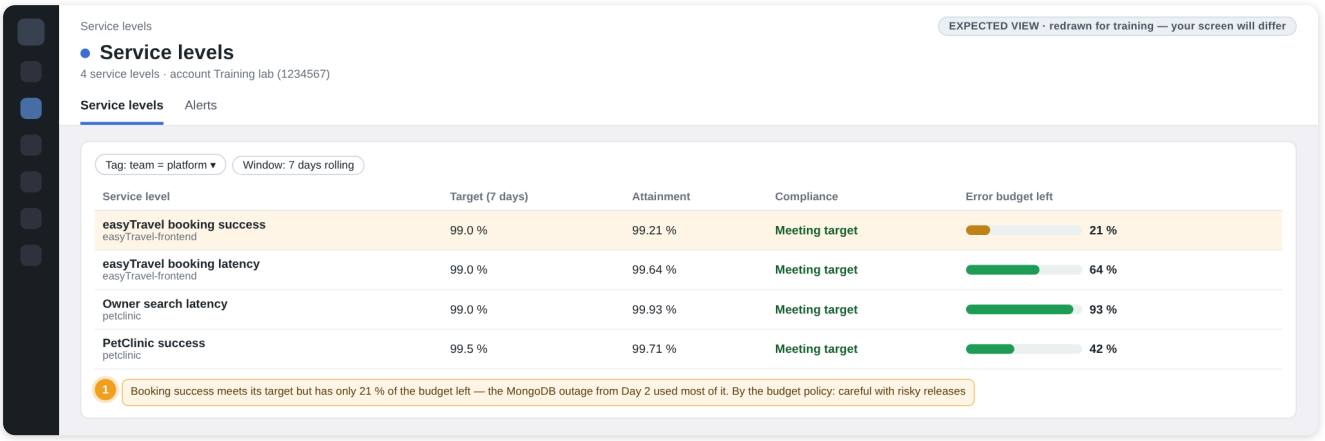
```
mutation { serviceLevelCreate(entityGuid: "PETCLINIC_GUID", indicator: {
    name: "Owner search latency",
    description: "Share of owner searches answered in under 100 ms",
    events: { accountId: 1234567,
      validEvents: { from: "Transaction", where: "entityGuid = 'PETCLINIC_GUID' AND name LIKE
'%OwnerSearch%'" },
      goodEvents:  { from: "Transaction", where: "entityGuid = 'PETCLINIC_GUID' AND name LIKE
'%OwnerSearch%' AND duration < 0.1" } },
    objectives: [{ target: 99.0, timeWindow: { rolling: { count: 7, unit: DAY } } }] }) {
  id } }
```

4. Add the PetClinic success SLI in the UI, as in 1.



Expected view — creating the latency service level: valid and good event queries, the target and the 7-day window with a feasibility preview

After a few minutes, the **Service levels** list shows each SLI with its attainment, target and error budget left.

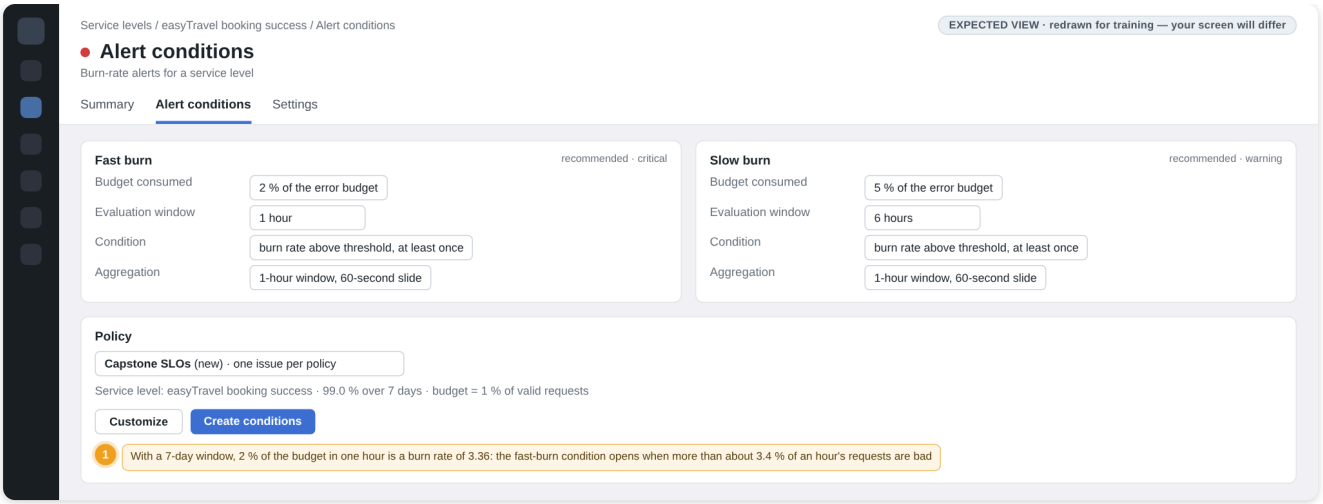


Expected view — the service levels list: attainment, target, compliance and error budget left for the four SLIs

Question 1. The booking success SLI shows 99.21 % against a 99 % target over 7 days. How much of the error budget is left?

Step 4 — Burn-rate alerts into the existing routing

For each service level: **Settings** → **Alert conditions** → **Alert**, keep the recommended **fast-burn** (2 % of the budget in 1 hour) and **slow-burn** (5 % in 6 hours) conditions, and put them in a new policy `Capstone SLOs`. Add that policy to the Slack workflow filter from Lab 5.2 (*policy is* `PetClinic synthetics` *OR* `Capstone SLOs`).



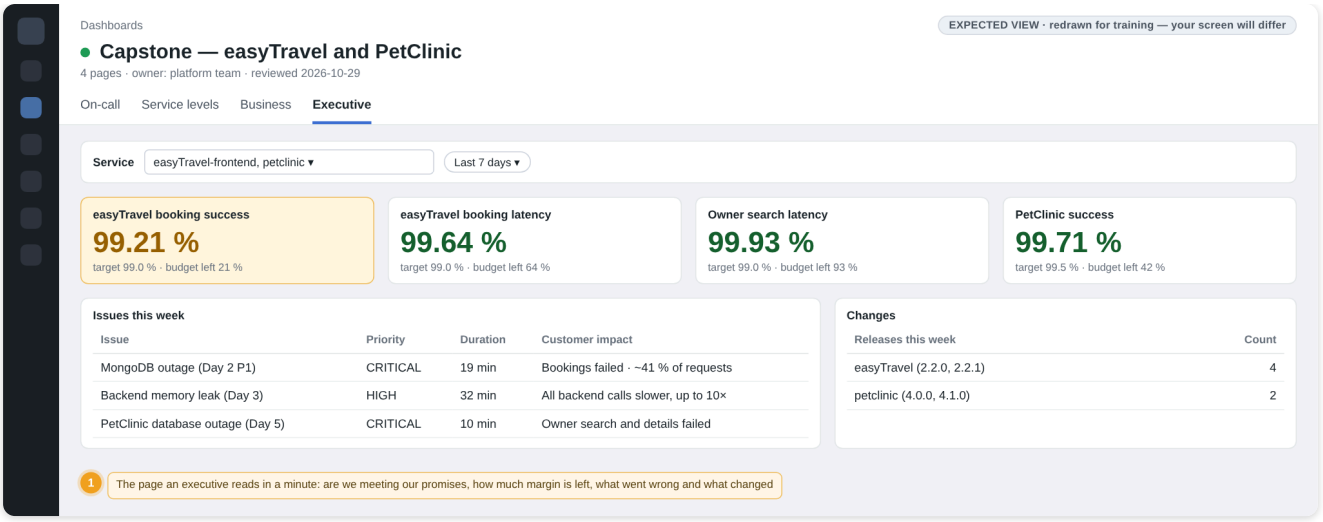
Expected view — the burn-rate alert panel: fast-burn and slow-burn conditions with their thresholds, into the Capstone SLOs policy

With a 7-day window (168 hours), 2 % of the budget in one hour is a burn rate of 0.02 × 168 = 3.36: with a 1 % budget, the fast-burn condition opens when more than about 3.4 % of an hour's requests are bad.

Step 5 — Service-level, business and executive pages

Add three pages to the Day 3 dashboard `[Support] easyTravel + PetClinic – On-call`, or create a new dashboard `Capstone – easyTravel and PetClinic`:

Page	Widget	Query (or source)
Service levels	SLI attainment per SLI	`FROM Metric SELECT clamp_max((count(newrelic.sli.valid) - count(newrelic.sli.bad)) / count(newrelic.sli.valid) * 100, 100) WHERE sli.id = 'SLI_ID' TIMESERIES AUTO` (one widget per SLI; the SLI's page shows its query)
Service levels	Error budget left	Add from each service level's summary page (*Add to dashboard*)
Business	Owner searches by outcome	`SELECT count(*) FROM OwnerSearch FACET outcome TIMESERIES`
Business	Find-owner funnel	The funnel from Lab 5.1, Question 4
Business	Core Web Vitals	`SELECT percentile(largestContentfulPaint, 75) FROM PageViewTiming WHERE appName = 'petclinic' AND timingName = 'largestContentfulPaint' TIMESERIES`
Executive	7-day availability per journey	Billboards from the service levels, thresholds at the targets
Executive	Issues this week	`SELECT uniqueCount(issueId) FROM NrAiIssue FACET priority SINCE 7 days ago`
Executive	Releases this week	`SELECT count(*) FROM Deployment SINCE 7 days ago`



Expected view — the executive page: availability against target per journey, error budget left, issues and releases this week

Step 6 — The one-page performance report

```
mkdir -p ~/nr-lab/capstone
cat > ~/nr-lab/capstone/report.md <<'EOF'
# Performance report — easyTravel and PetClinic — week of <date>

## 1. Service levels (7-day rolling)
| Journey | SLI | Target | Attained | Budget left | Trend |
| --- | --- | --- | --- | --- | --- |
| Book a trip | Success | | | | |
| Book a trip | Latency < __ s | | | | |
| Find an owner | Latency < 100 ms | | | | |
| Use PetClinic | Success | | | | |

## 2. Incidents
| When | What users saw | Cause | Time to acknowledge | Time to resolve | Budget used |
| --- | --- | --- | --- | --- | --- |

## 3. Changes
Releases this week, and their effect on the SLIs:

## 4. Top three recommendations
1.
2.
3.

## 5. Decisions needed
EOF
```

Fill it from the dashboard and the saved queries — every number must come from a query someone else can rerun. Use the incidents you caused this week (MongoDB outage, the memory leak, PetClinic's database outage) as the incident list.

Step 7 — Three recommendations, with evidence

Choose three findings — from the list below or your own — and write each in the six-part structure from the study guide (15.4): **observation, impact, cause, recommendation, effort and risk, verification**. Attach one chart or query to each.

Finding from the course	Evidence you have
MongoDB is a single point of failure for every easyTravel booking	Lab 2.3 P1 timeline; error rate 41 %; one issue, all journeys affected
The backend leaks memory under location search	Lab 3.1 heap floor, GC time and response time rising together
Owner search was 30× slower than needed	Lab 3.2 explain plan and the before/after with the index
The vet list page fails for half of the visitors in the browser	Lab 4.2 JavaScript errors, invisible in APM
A fifth of the browser visitors' owner searches find nothing	Lab 5.1 funnel and `OwnerSearch` events
The static latency alert paged five times for one problem	Lab 2.2 `NrAiIncident` comparison

Rank them by impact against effort and present: 5 minutes, one slide or dashboard page per recommendation, ending with the decision you need.

Capstone review criteria

Criterion	Evidence
Setup is complete and consistent	Audit queries; tags; every service traced, versioned, with logs
Service levels are meaningful	Targets chosen from data; valid and good events match the journey; burn-rate alerts routed
Dashboards serve their readers	On-call, service-level, business and executive pages; thresholds match alerts
Report is reproducible	Every number from a saved query; incidents with cause and budget impact
Recommendations are evidence-based	Six-part structure; a chart each; verification stated

Check your work

- ☐ Four service levels exist with targets from data, and burn-rate conditions in `Capstone SLOs`, routed to Slack.
- ☐ The dashboard has service-level, business and executive pages.
- ☐ `report.md` is filled in, and three recommendations are ready to present.

Answer. 1 — The budget is 100 % − 99 % = 1 % of requests. 100 % − 99.21 % = 0.79 % were bad, so 79 % of the budget is used and **21 %** is left — under 25 %: by a typical budget policy, time to slow down risky releases.

Appendix — Reset, troubleshooting and course close-out

Reset and clean-up

```
kind delete cluster --name nr-lab
docker stop pc-browser synthetics-job-manager
pkill -f pcload.sh || true
docker start petclinic-db
docker ps --format "table {{.Names}}\t{{.Status}}"
```

In New Relic: disable the `PetClinic to Slack` workflow (and the PagerDuty and ServiceNow workflows), the `PetClinic synthetics` and `Capstone SLOs` policies and the monitors in shared accounts; delete the service levels if the account is shared. In Slack, remove the New Relic app from the training workspace when the course is over.

Troubleshooting

Symptom	Check	Fix
Funnel shows no drop between steps	`docker logs --tail 10 pc-browser`	The new script is not running: `docker restart pc-browser`; wait 20 minutes
`capture()` returns nothing	The regular expression	It must match the whole value and use a named group `(?P<name>…)`
Slack test message does not arrive	Destination status	Reauthorise the Slack destination; the app must be allowed in the workspace and added to the channel
`deploy.sh` fails at *record deployment*	`echo $GUID`	Empty GUID: wrong key or region; the user key needs NerdGraph access to the account
Marker created, no version on transactions	`docker exec petclinic env`	The compose file was not changed or the container was not recreated (`--force-recreate`)
kind pods stay `Pending`	`kubectl describe pod -n newrelic …`	Not enough memory: stop the headless browsers and the job manager while you work on Kubernetes
No `K8s*` data	`kubectl logs -n newrelic -l app.kubernetes.io/name=newrelic-infrastructure`	License key empty (source `.env` before `helm`); cluster name typo in queries
Two issues instead of one	Policy settings and Decisions	*Correlate and suppress noise* off, or incidents too far apart in time
`serviceLevelCreate` returns an error	The `where` clauses	Use the entity GUID in both queries; the account ID must be the entity's account
SLI shows *no data*	Wait 5–10 minutes	SLI metrics are computed from new data only; widen the window after the first hour

End-of-course checklist

- ☐ All load generators, browsers, the job manager and the kind cluster stopped or removed.
- ☐ Workflows, policies, monitors, service levels and dashboards disabled, deleted or handed over in shared accounts.
- ☐ License key, user key, private location key and any PagerDuty or ServiceNow credentials removed from the VM or rotated.
- ☐ Your queries, dashboards (JSON), scripts, `report.md` and recommendations saved outside the lab VM.

Sources

- New Relic — NRQL syntax, clauses and functions
- New Relic — Create service levels
- New Relic — Alerting on service levels
- New Relic — NerdGraph service level examples
- New Relic — Change tracking with NerdGraph
- New Relic — Kubernetes integration Helm example
- kind — Quick start